CM12002
Computer Systems Architectures

Sequential Logic

Fabio Nemetz

Sequential Logic

Summary

In this lecture:

- We introduce sequential logic circuits, considering the behaviour of a simple latch circuit and flip-flops

Sequential logic

Logic systems with feedback

- The output of a logic gate in a *combinational* logic system may form the input to a gate ‘further down’ the system.

In a *sequential* system, the output of a logic gate can also be *fed back* so as to become the input to a gate ‘earlier’ in the system

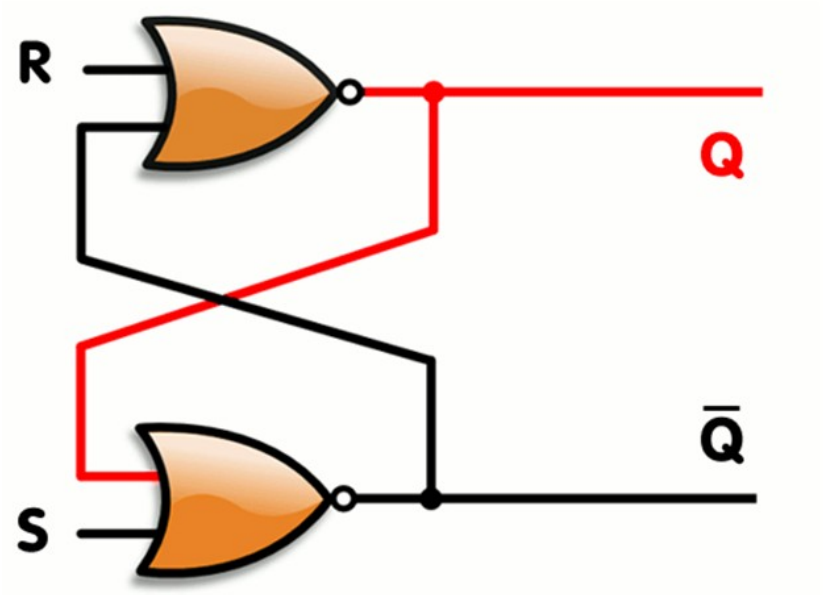
- This means that the output may depend not only on the current input, but on previous inputs **--- the system has *state.***

(reference: Stallings book, chapter 12, section 4)

SR Latch (Set-Reset latch)

Input combinations may result in any one of a number of stable or unstable states --- the system may be *nondeterministic.*

A simple SR (**set-reset**) latch can be implemented with NOR gates:

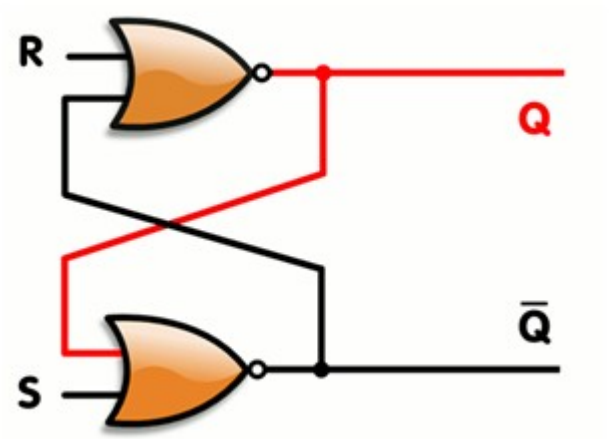


NOR: truth table

input	output
00	1
01	0
10	0
11	0

SR Latch (Set-Reset Latch)

- 2 inputs: **S** (Set) and **R** (Reset)
- 2 outputs: Q and Q’ (same as $\bar{Q}$) (complements)
- 2 NOR gates connected in a feedback way
- Q is the value we’re interested in
- But how does it work?



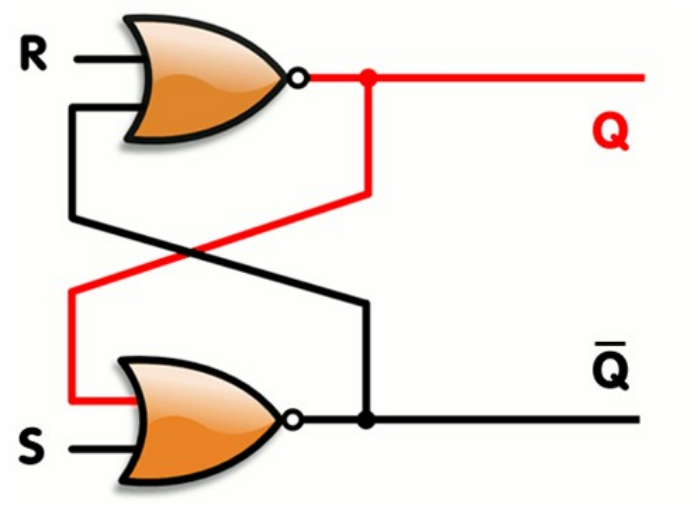
NOR: truth table

input	output
00	1
01	0
10	0
11	0

SR Latch

- Let's start with **S = R = 0, and Q = 0**
- Lower NOR gate inputs will be Q=0 and S=0 → Q'=1
- Upper NOR gate inputs: R=0 and Q'= 1 → Q = 0
- This is consistent and **stable for S=R=0**

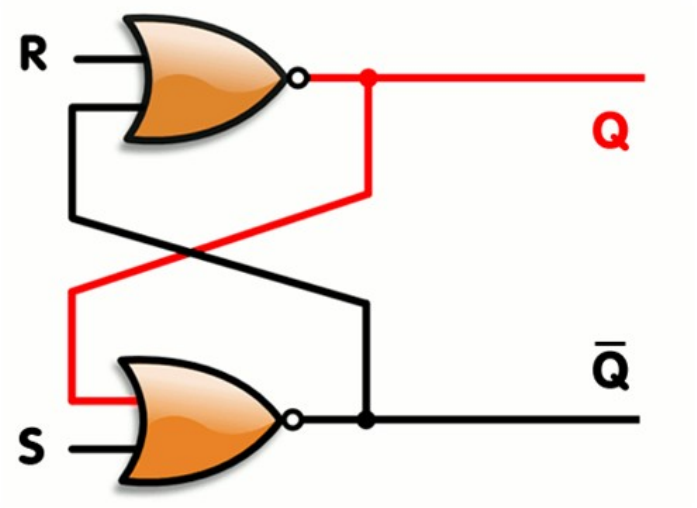
- The same is valid for **S = R = 0, and Q = 1**



This circuit can function as a 1-bit memory: **Q** is the value of the bit

SR Latch: Set

- Let's start again with **S = R = 0, and Q = 0** (we know that Q' = 1)
- Now **S** (Set) changes to **1**
- Lower NOR gate inputs will be Q=0 and S=**1** → Q'= 0
- Upper NOR gate inputs: R=0 and Q'= **0** → Q = **1**
- This is consistent and stable for S=1, R=0

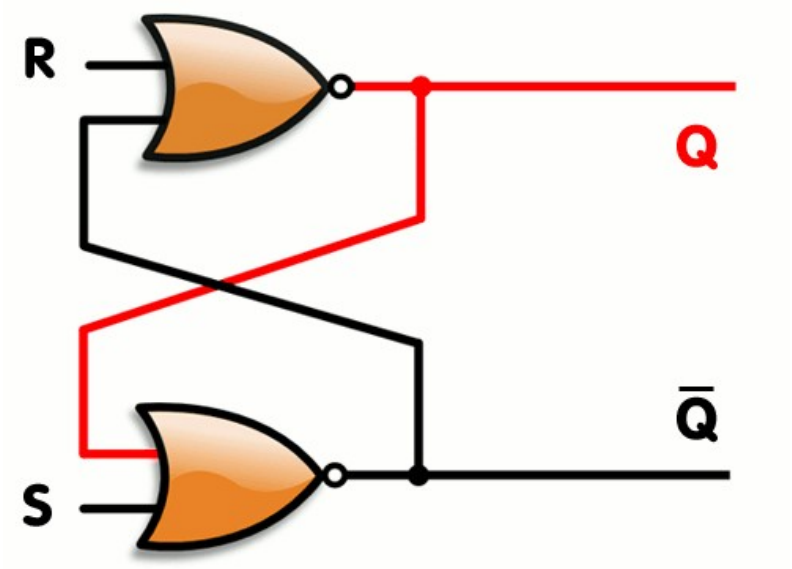


NOR: truth table

input	output
00	1
01	0
10	0
11	0

SR Latch: Reset

- Let's assume **S = R = 0, and Q = 1** (we know that Q' = 0)
- Now **R** (Reset) changes to **1**
- Upper NOR gate inputs will be R=**1** and Q'=0 → **Q= 0**
- Lower NOR gate inputs: Q=0 and S=0 → Q' = **1**
- This is consistent and stable for S=0, R=1

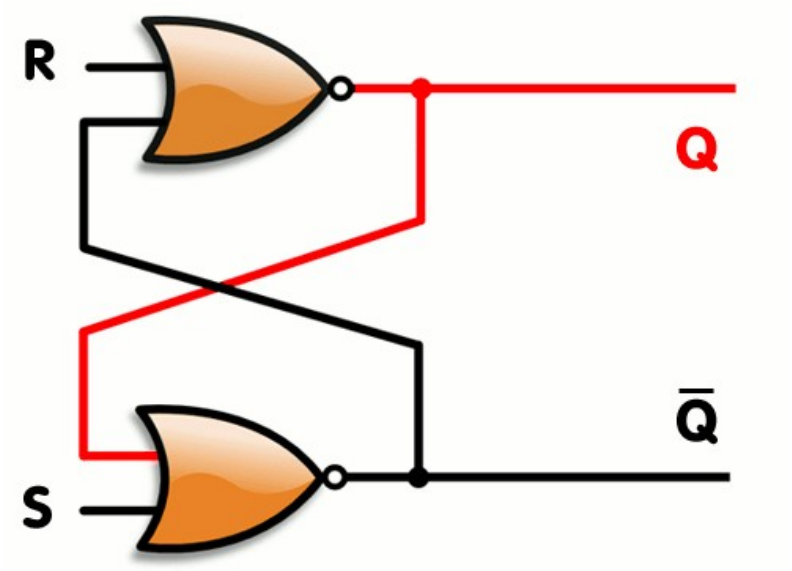


NOR: truth table

input	output
00	1
01	0
10	0
11	0

SR Latch

- What happens if S=1 AND R=1 ????
- These inputs are not allowed, because these would produce an inconsistent output (both Q and Q’ equal 0).



NOR: truth table

input	output
00	1
01	0
10	0
11	0

SR Latch

Given inputs S and R, and current Q, what's the next state?

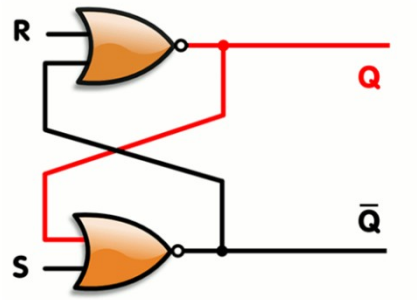
Truth tables, in this case, are also called characteristic table

(a) Characteristic Table

Current Inputs	Current State	Next State
SR	Q_n	Q_{n+1}
00	0	0
00	1	1
01	0	0
01	1	0
10	0	1
10	1	1
11	0	–
11	1	–

(b) Simplified Characteristic Table

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	–



SR Latch

- We can see that time and sequence are involved in this process:
  - S pulses to set Q; after some delay, Q = 1

- There is a *delay* before a logic gate responds to an input;
- Two separate gates cannot switch simultaneously

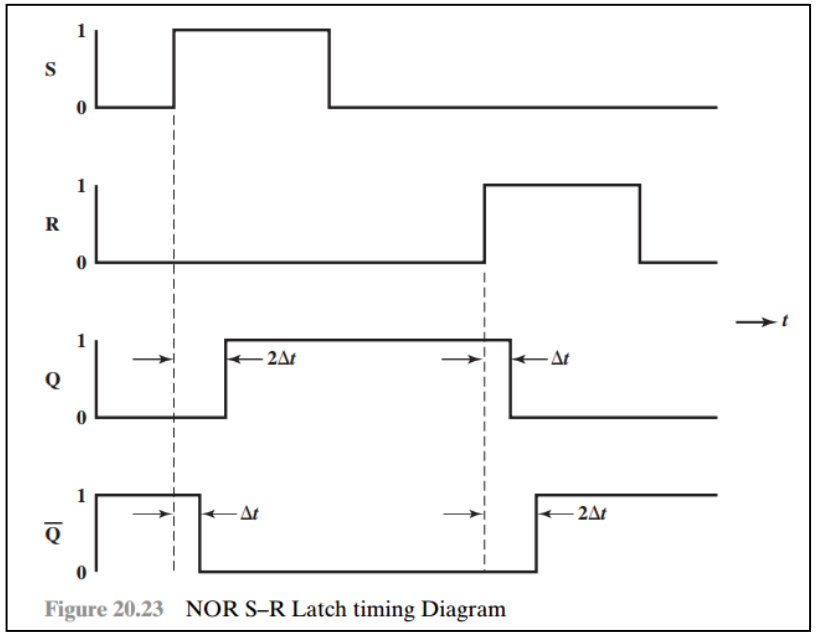
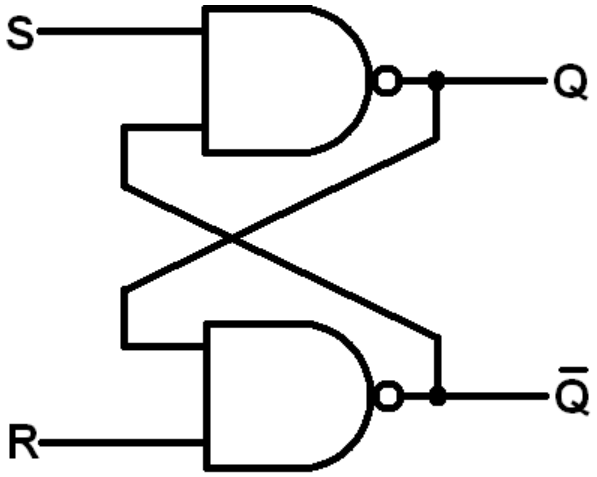


Figure 20.23 NOR S–R Latch timing Diagram

SR latches are asynchronous or unclocked

Inverted SR Latch

- It is possible to implement an inverted SR latch using NAND gates
  - you'll see this implementation in some books and even in our lecture notes
- In this case it Sets when S=0; Resets when R=0
  - can't have R=S=0



Redo characteristic tables

Current inputs	Current state	Next state
SR	Q_n	Q_{n+1}
00		
00		
01		
01		
10		
…		

Problems with simple SR Latches

We may observe two problems:

- Asynchronous transitions:
  - in circuits with several sequential components, output states may depend on the order in which changes to inputs occur, or the moment the circuit is tested.

  Solution: use a *clock* input to enable/disable transitions, yielding a *gated latch or flip-flop.*

- Non-deterministic transitions associated with certain inputs

  Solution: avoid states leading to such transitions by *adding gates to control the inputs.*

Flip-flops

Flip-flops introduce a **clock** to latches: this synchronises state transitions across sequential logic circuits

We'll see 3 types of flip-flops: SR, D and J-K

SR flip-flop

AKA Clocked SR flip-flop or Gated SR latch

The simple latch can be extended to be a more useful control circuit by adding a *clock* signal so that the latch responds to inputs ***only*** **when the clock signal is present, rather than at** ***any*** **time.**

R and S inputs are passed to the NOR gates only during the clock pulse.

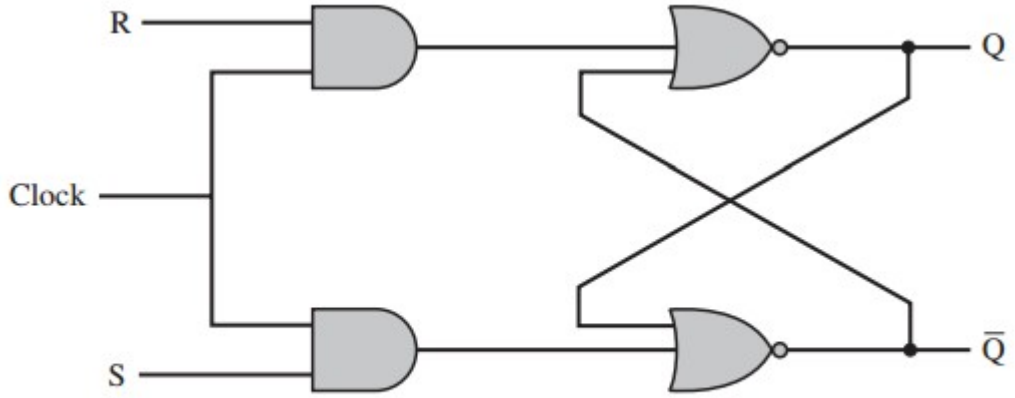


Figure 20.24 Clocked S–R Flip Flop

D-type flip-flop (D for data or delay)

- One problem with S–R flip-flop is that the condition R=1, S=1 must be avoided. One way to do this is to ensure that the two inputs are always different. This is ensured in the D-type flip-flop.
- By using an inverter, the nonclock inputs to the two AND gates are guaranteed to be the opposite of each other.

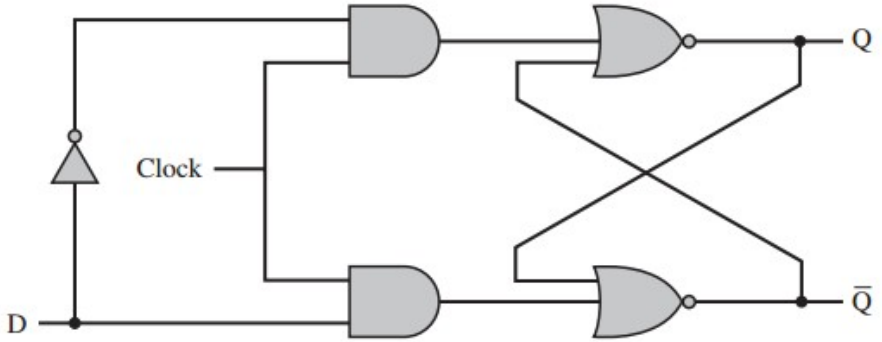


Figure 20.25 D Flip Flop

D	Q_{n+1}
0	0
1	1

- The D flip-flop is referred to as the **data flip-flop** because it is, in effect, storage for one bit of data. The output of the D flip-flop is always equal to the most recent value applied to the input. Hence, it remembers and produces the last input.
- It is also referred to as **the delay flip-flop**, because it delays a 0 or 1 applied to its input for a single clock pulse.

J-K flip-flop

This type of flip-flop makes use of the restricted combination (1,1). It is used to *toggle*, or *change* the output state, of the flip-flop.

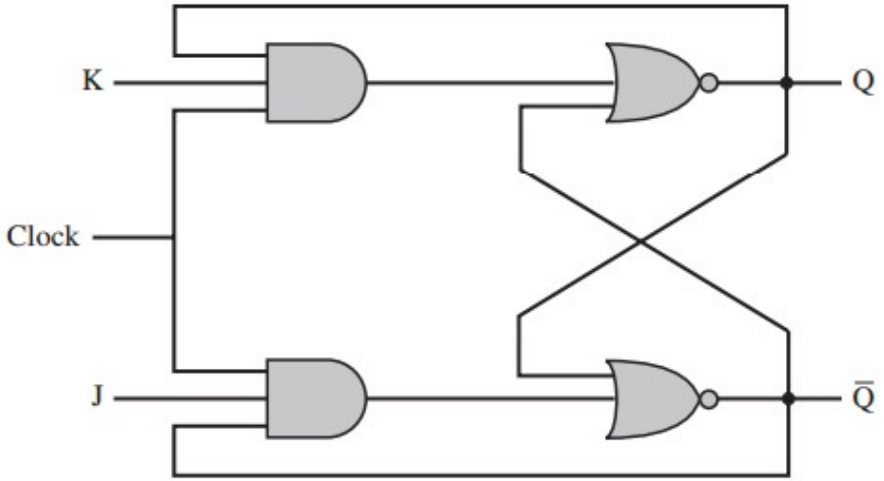
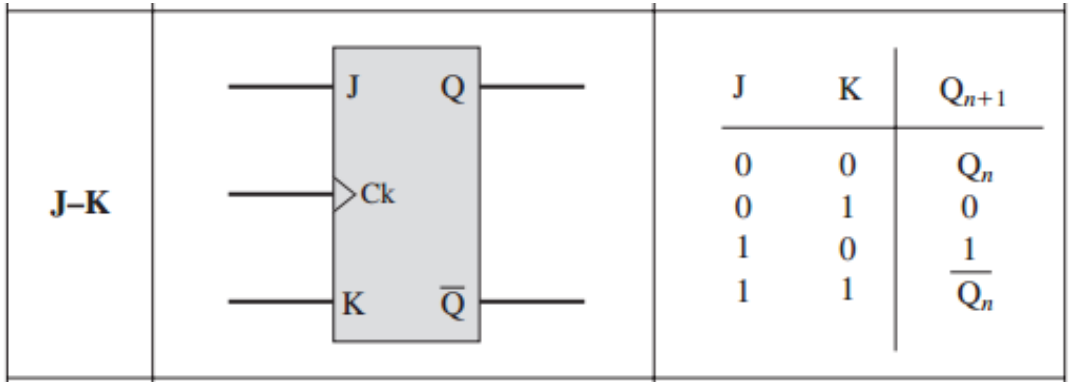


Figure 20.26 J–K Flip Flop



Flip-flops (summary)

S-R

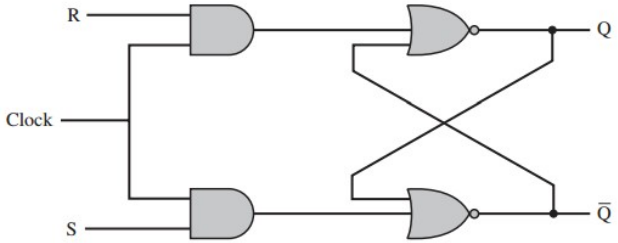


Figure 20.24 Clocked S–R Flip Flop

J-K

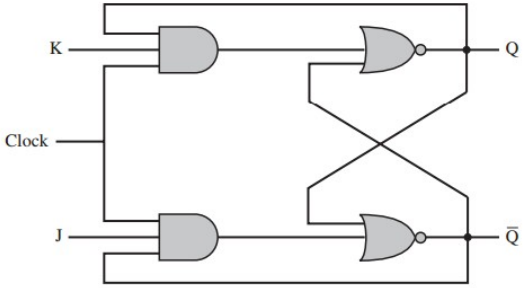


Figure 20.26 J–K Flip Flop

D

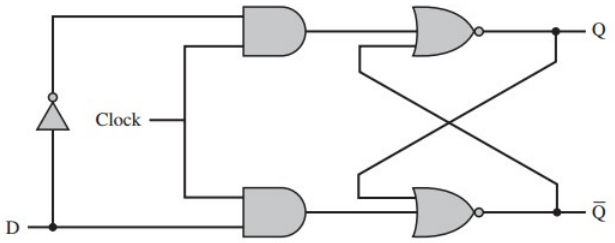


Figure 20.25 D Flip Flop

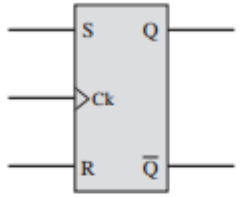
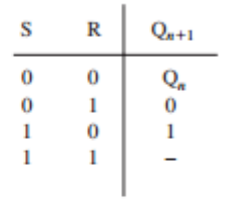
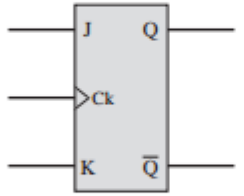
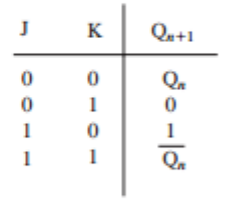
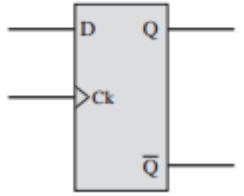
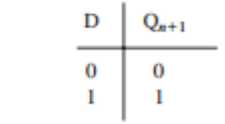
Name	Graphical Symbol	Truth Table
S–R		
J–K		
D		

Figure 20.27 Basic Flip-Flops

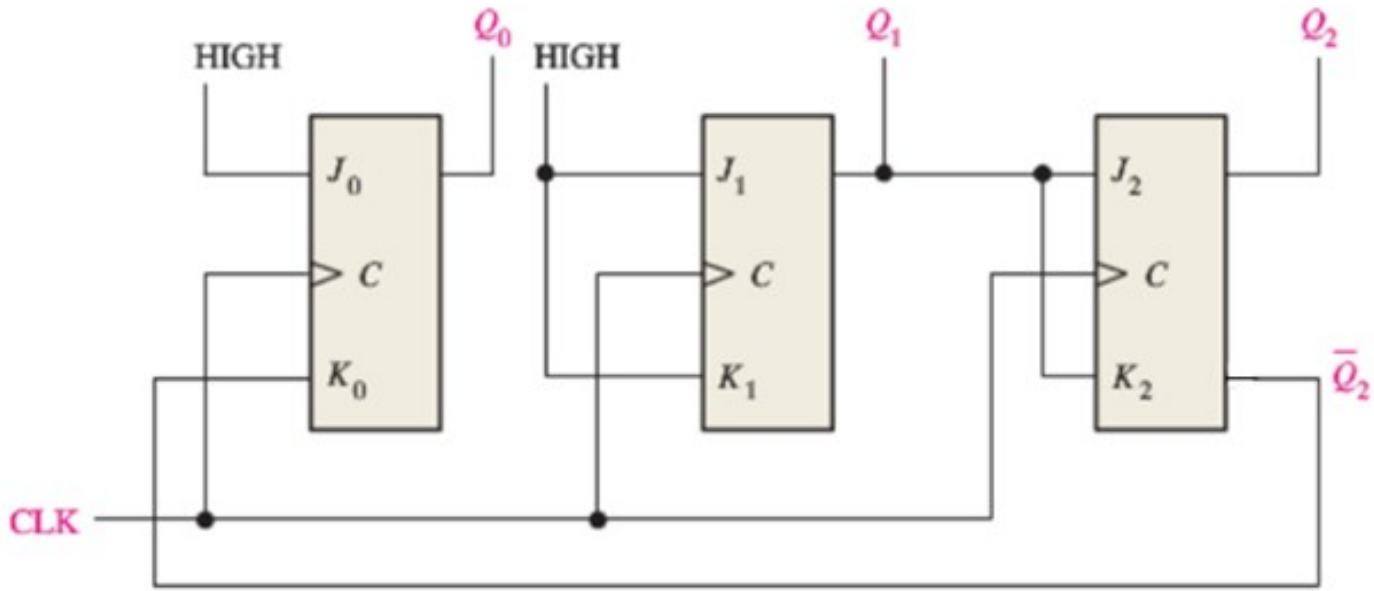
The J-K flip-flop is a *universal* flip-flop:

- it can behave as an SR flip-flop or D flip-flop.

Homework

2.

(a) The circuit below uses 3 J-K flip-flops to create an irregular binary counter. Starting with 001 (Q_2=0, Q_1=0, Q_0=1), what are the next 4 3-bit sequences for each clock cycle?



Next time --- using flip-flops